PYTHON ACTIVITY-BASED LEARNING REPORT

Name: Thanuj Lanke | Roll No: 26000175 | Branch: CSE | Course: Python Programming Lab

Case 1: Progressive Electricity Billing

PROBLEM STATEMENT

Write a Python program to calculate the progressive electricity bill based on units consumed using tiered pricing rules.

ALGORITHM

- Step 1: Start.
- Step 2: Read units consumed from user.
- Step 3: If units ≤ 100, bill = units × 3.0.
- Step 4: Else if units ≤ 300, bill = (100 × 3.0) + ((units - 100) × 5.0).
- Step 5: Else bill = (100 × 3.0) + (200 × 5.0) + ((units - 300) × 8.0).
- Step 6: Print total bill and Stop.

FLOWCHART (MERMAID REPRESENTATION)

```
[Start] → [Input units] → {units ≤ 100?}
• Yes → [bill = units * 3.0] → [Print bill]
• No → {units ≤ 300?} → [Calculate Tier 2] → [Print bill]
• Else → [Calculate Tier 3] → [Print bill] → [Stop]
```

SOURCE CODE

```
# c-1 Progressive Electricity Billing
units = int(input("Enter units consumed: "))

if units <= 100:
    bill = units * 3.0
elif units <= 300:
    bill = (100 * 3.0) + ((units - 100) * 5.0)
else:
    bill = (100 * 3.0) + (200 * 5.0) + ((units - 300) * 8.0)

print("Total Electricity Bill: Rs.", bill)
```

SAMPLE INPUT & OUTPUT

```
Enter units consumed: 350
Total Electricity Bill: Rs. 1700.0
```

Case 2: Employee Payroll Analysis

PROBLEM STATEMENT

Write a Python program to calculate gross salary, tax deductions based on income slab, and net salary for an employee.

ALGORITHM

- Step 1: Start.
- Step 2: Read basic pay from user.
- Step 3: Calculate allowance = basic * 0.20 and gross = basic + allowance.
- Step 4: If gross > 40000, tax = gross * 0.10, else tax = 0.
- Step 5: Calculate net = gross - tax and print salaries. Stop.

FLOWCHART (MERMAID REPRESENTATION)

```
[Start] → [Input basic] → [Calculate gross] → {gross > 40000?}
                  • Yes → [tax = gross * 0.10]
                        • No → [tax = 0]
   [Calculate net = gross - tax] → [Print results] → [Stop]
```

SOURCE CODE

```
# c-4 Employee Payroll Analysis
basic = float(input("Enter basic pay: "))
allowance = basic * 0.20
gross = basic + allowance

tax = gross * 0.10 if gross > 40000 else 0
net = gross - tax

print("Gross Salary:", gross)
print("Tax Deducted:", tax)
print("Net Salary:", net)
```

SAMPLE INPUT & OUTPUT

```
Enter basic pay: 10000
Gross Salary: 12000.0
Tax Deducted: 0
Net Salary: 12000.0
```

Case 3: Railway Fare Calculator

PROBLEM STATEMENT

Write a Python program to calculate railway fare based on distance with a 25% discount for children (under 12) and senior citizens (above 60).

ALGORITHM

- Step 1: Start.
- Step 2: Read distance and passenger age.
- Step 3: Calculate base_fare = distance * 1.5.
- Step 4: If age < 12 or age > 60, apply 25% discount (base_fare *= 0.75).
- Step 5: Print final railway fare. Stop.

FLOWCHART (MERMAID REPRESENTATION)

```
[Start] → [Input distance, age] → [base_fare = distance * 1.5] → {age <
                                  12 or age > 60?}
                        • Yes → [base_fare = base_fare * 0.75]
                                  • No → [No discount]
                               [Print final fare] → [Stop]
```

SOURCE CODE

```
# c-6 Railway Fare Calculator
distance = int(input("Enter distance in km: "))
age = int(input("Enter passenger age: "))

base_fare = distance * 1.5
if age < 12 or age > 60:
    base_fare = base_fare * 0.75  # 25% discount

print("Final Railway Fare: Rs.", base_fare)
```

SAMPLE INPUT & OUTPUT

```
Enter distance in km: 69
Enter passenger age: 19
Final Railway Fare: Rs. 103.5
```

Case 4: Smart Parking Fee

PROBLEM STATEMENT

Write a Python program to calculate total parking fees depending on vehicle type (Bike or Car) and duration in hours.

ALGORITHM

- Step 1: Start.
- Step 2: Read hours and vehicle_type.
- Step 3: Set rate = 10 if vehicle is bike else 20.
- Step 4: Calculate total_fee = hours * rate and print. Stop.

FLOWCHART (MERMAID REPRESENTATION)

```
[Start] → [Input hours, vehicle_type] → {vehicle_type == "bike"?}
                        • Yes → [rate = 10]
                        • No → [rate = 20]
      [total_fee = hours * rate] → [Print total fee] → [Stop]
```

SOURCE CODE

```
# c-15 Smart Parking Fee
hours = int(input("Enter parking duration in hours: "))
vehicle_type = input("Enter vehicle type (Bike/Car): ").lower()

rate = 10 if vehicle_type == "bike" else 20
total_fee = hours * rate

print("Total Parking Fee: Rs.", total_fee)
```

SAMPLE INPUT & OUTPUT

```
Enter parking duration in hours: 3
Enter vehicle type (Bike/Car): bike
Total Parking Fee: Rs. 30
```

Case 5: Traffic Violation Penalty

PROBLEM STATEMENT

Write a Python program to calculate traffic violation fines with double penalty for repeat offenses.

ALGORITHM

- Step 1: Start.
- Step 2: Read offence type and repeat status.
- Step 3: Determine base fine (Helmet: 500, Speeding: 1500, Drunk: 10000).
- Step 4: If repeat == "yes", double the fine. Print total fine. Stop.

FLOWCHART (MERMAID REPRESENTATION)

```
[Start] → [Input offence, repeat] → [Determine base fine] → {repeat ==
                                "yes"?}
                        • Yes → [fine = fine * 2]
                           • No → [No doubling]
                  [Print total penalty fine] → [Stop]
```

SOURCE CODE

```
# c-21 Traffic Violation Penalty
offence = input("Enter offence (Helmet / Speeding / Drunk): ")
repeat = input("Is it a repeat offence? (yes/no): ")

fine = 500 if offence == "Helmet" else (1500 if offence == "Speeding"
else 10000)
if repeat == "yes":
    fine *= 2

print("Total Penalty Fine: Rs.", fine)
```

SAMPLE INPUT & OUTPUT

```
Enter offence (Helmet / Speeding / Drunk): Helmet
Is it a repeat offence? (yes/no): no
Total Penalty Fine: Rs. 500
```

Case 6: Courier Charge Calculator

PROBLEM STATEMENT

Write a Python program to calculate courier charges based on parcel weight and urgency status.

ALGORITHM

- Step 1: Start.
- Step 2: Read parcel weight and urgency.
- Step 3: Set rate = 50 if weight ≤ 5 else 100, calculate total = weight * rate.
- Step 4: If urgent == "yes", add 200 surcharge. Print total. Stop.

FLOWCHART (MERMAID REPRESENTATION)

```
  [Start] → [Input weight, urgent] → {weight ≤ 5?} → [Calculate rate &
                    total] → {urgent == "yes"?}
                       • Yes → [total += 200]
                [Print total courier charges] → [Stop]
```

SOURCE CODE

```
# c-24 Courier Charge Calculator
weight = float(input("Enter parcel weight in kg: "))
urgent = input("Is it urgent? (yes/no): ")

rate = 50 if weight <= 5 else 100
total = weight * rate

if urgent == "yes":
    total += 200

print("Total Courier Charges: Rs.", total)
```

SAMPLE INPUT & OUTPUT

```
Enter parcel weight in kg: 30
Is it urgent? (yes/no): yes
Total Courier Charges: Rs. 3200.0
```

Case 7: ATM Menu Simulation

PROBLEM STATEMENT

Write a Python program simulating an ATM menu allowing balance check, deposit, and withdrawal options.

ALGORITHM

- Step 1: Start. Initialize balance = 5000.0.
- Step 2: Display menu and read user choice (1-3).
- Step 3: If choice == 1, display balance.
- Step 4: If choice == 2, read deposit amount and update balance.
- Step 5: If choice == 3, read withdrawal amount and update balance. Stop.

FLOWCHART (MERMAID REPRESENTATION)

```
      [Start] → [Display Menu & Input Choice] → {Choice == 1/2/3?}
                          • 1 → [Show Balance]
                         • 2 → [Deposit & Update]
                        • 3 → [Withdraw & Update]
                                  [Stop]
```

SOURCE CODE

```
# c-38 ATM Menu Simulation
balance = 5000.0
print("1. Check Balance\n2. Deposit\n3. Withdraw")
choice = int(input("Enter choice (1-3): "))

if choice == 1:
    print("Balance:", balance)
elif choice == 2:
    dep = float(input("Enter deposit amount: "))
    balance += dep
    print("New Balance:", balance)
elif choice == 3:
    wd = float(input("Enter withdrawal amount: "))
    if wd <= balance:
        balance -= wd
        print("New Balance:", balance)
    else:
        print("Insufficient balance!")
```

SAMPLE INPUT & OUTPUT

```
1. Check Balance
2. Deposit
```

```
3. Withdraw
Enter choice (1-3): 1
Balance: 5000.0
```

Case 8: Sports Tournament Points

PROBLEM STATEMENT

Write a Python program to compute total tournament points given matches won and drawn (Win = 3 points, Draw = 1 point).

ALGORITHM

- Step 1: Start.
- Step 2: Read matches won and matches drawn.
- Step 3: Calculate points = (wins * 3) + (draws * 1).
- Step 4: Print total tournament points. Stop.

FLOWCHART (MERMAID REPRESENTATION)

```
[Start] → [Input wins, draws] → [points = (wins * 3) + (draws * 1)] →
                        [Print points] → [Stop]
```

SOURCE CODE

```
# c-47 Sports Tournament Points
wins = int(input("Enter total matches won: "))
draws = int(input("Enter total matches drawn: "))

points = (wins * 3) + (draws * 1)
print("Total Tournament Points:", points)
```

SAMPLE INPUT & OUTPUT

```
Enter total matches won: 5
Enter total matches drawn: 1
Total Tournament Points: 16
```

Case 9: Transaction Audit using Tuples

PROBLEM STATEMENT

Write a Python program to audit financial transactions stored as tuples and check for anomalies.

ALGORITHM

- Step 1: Start.
- Step 2: Define a transaction tuple containing txn_id, amount, and txn_type.
- Step 3: Unpack tuple variables.
- Step 4: Check if amount < 0. If yes, print warning, else print valid transaction record. Stop.

FLOWCHART (MERMAID REPRESENTATION)

```
[Start] → [Define tuple & unpack] → {amount < 0?}
                • Yes → [Print Warning]
               • No → [Print Valid Record]
                        [Stop]
```

SOURCE CODE

```
# c-55 Transaction Audit using Tuples
transaction = ("TXN101", 1500.0, "DEPOSIT")
txn_id, amount, txn_type = transaction

if amount < 0:
    print("Warning: Negative amount detected in", txn_id)
else:
    print(f"Valid {txn_type} of Rs.{amount} recorded for {txn_id}.")
```

SAMPLE INPUT & OUTPUT

```
Valid DEPOSIT of Rs.1500.0 recorded for TXN101.
```

Case 10: Library Fine Calculator

PROBLEM STATEMENT

Write a Python program to calculate library overdue fines based on allowed days and actual days kept.

ALGORITHM

- Step 1: Start. Set allowed_days = 14.
- Step 2: Read actual_days kept from user.
- Step 3: If actual_days > allowed_days, calculate fine = (actual_days - allowed_days) * 5 and print fine.
- Step 4: Else, print book returned on time. Stop.

FLOWCHART (MERMAID REPRESENTATION)

```
[Start] → [Input actual_days] → {actual_days > 14?}
• Yes → [fine = (actual_days - 14) * 5] → [Print fine]
• No → [Print no fine]
[Stop]
```

SOURCE CODE

```
# c-58 Library Fine Calculator
allowed_days = 14
actual_days = int(input("Enter total days kept: "))

if actual_days > allowed_days:
    fine = (actual_days - allowed_days) * 5
    print("Book is overdue. Fine to pay: Rs.", fine)
else:
    print("Book returned on time. No fine!")
```

SAMPLE INPUT & OUTPUT

```
Enter total days kept: 30
Book is overdue. Fine to pay: Rs. 80
```